U N I V E R S I D A D E D A M A D E I R A

Engenharia de Software

Madflix

Juan Olim – 2030418
Luís Andrade - 2131023
Manuel Fernandes - 2112323
Ricardo Nunes – 2118823

2025/2026

Índice

Introdução 3
Tecnologias Utilizadas 4
Estrutura do Projeto 5
Funcionalidades Implementadas 7
Esquema Relacional da Base de Dados 10
Padrões de Desenho Aplicados 11

Introdução

O projeto MadFlix consiste numa aplicação web desenvolvida no âmbito académico, com o objetivo de simular uma plataforma online dedicada a festivais de cinema. A aplicação permite aos utilizadores consultar filmes, explorar festivais, comprar diferentes tipos de acesso e interagir com conteúdos através de comentários, avaliações e listas pessoais.

A plataforma foi pensada para representar um sistema de festivais de cinema digital, onde existem filmes individuais e filmes associados a festivais. Dependendo do tipo de acesso adquirido, o utilizador pode visualizar determinados filmes dentro de janelas temporais específicas. O sistema distingue ainda utilizadores comuns e administradores, permitindo que apenas administradores tenham acesso a funcionalidades de gestão, como criação de festivais e associação de filmes a festivais.

Tecnologias Utilizadas

O sistema foi desenvolvido com base no ecossistema **.NET**, utilizando uma abordagem server-side através de **Blazor Web App**.

A camada de apresentação foi implementada com **Blazor Web App Server-Side**, permitindo criar interfaces web interativas usando C#. Esta tecnologia permite que a lógica da interface e a lógica da aplicação estejam fortemente integradas, evitando a necessidade de uma API REST separada para o funcionamento principal do projeto.

O projeto segue o caminho:

dotnet new blazor -n Madflix --interactivity Server

No nosso caso, o projeto foi desenvolvido como uma aplicação Blazor com interatividade no servidor. Isto significa que os componentes Razor são executados no servidor e a interface é atualizada no browser através da ligação interativa do Blazor.

Para a persistência de dados foi usado **Entity Framework Core**, em conjunto com **SQLite**. O EF Core funciona como ORM, permitindo mapear classes C# para tabelas da base de dados.

Foram utilizados pacotes como:

dotnet add package Microsoft.EntityFrameworkCore.Sqlite

dotnet add package Microsoft.EntityFrameworkCore.Design

As migrations foram usadas para criar e atualizar a estrutura da base de dados:

dotnet ef migrations add Inicial

dotnet ef database update

A aplicação é executada com:

dotnet run

Foram também usadas outras tecnologias e conceitos importantes:

- **ASP.NET Core Authentication** para login, registo e roles de utilizador.
- **Entity Framework Core Migrations** para controlo da evolução da base de dados.
- **SQLite** como base de dados relacional local.
- **Injeção de Dependências** para ligar serviços, repositórios, factories e componentes.
- **TMDB API** como fonte externa de dados sobre filmes.
- **JSON** para receber e tratar os dados devolvidos pela API externa.
- **CSS isolado e estilos Razor** para melhorar a apresentação visual das páginas.

Estrutura do Projeto

O projeto encontra-se organizado de forma a separar bem as responsabilidades de cada parte da aplicação. Esta organização facilita a manutenção do código, a evolução do sistema e a colaboração entre os vários elementos do grupo.

A aplicação segue uma separação por camadas, onde cada pasta tem uma função principal:

- Models/ contém as entidades de domínio da aplicação, ou seja, as classes que representam os principais conceitos do sistema, como utilizadores, filmes, festivais, carrinho, bilhetes, comentários e acessos.
- Services/ contém a lógica de negócio. É nesta camada que são tratadas as regras principais da aplicação, como criação de festivais, compra de acessos, validação de bilhetes, gestão do carrinho, comentários, listas pessoais e histórico de bilhetes.
- Repositories/ contém a lógica de acesso aos dados. Os repositórios são responsáveis por comunicar com a base de dados através do Entity Framework Core, permitindo consultar, adicionar, atualizar e remover informação.
- Data/ contém o AppDbContext, que faz a ligação entre os modelos da aplicação e a base de dados. É aqui que são definidos os DbSet e as relações entre as entidades.
- Components/Pages/ contém as páginas Blazor da aplicação, como a página inicial, página de festivais, detalhes dos filmes, bilhetes e perfil. Esta pasta representa a camada de apresentação, ou seja, aquilo com que o utilizador interage.
- Components/Layout/ contém componentes comuns da interface, como o menu de navegação, o layout principal, o carrinho lateral e as notificações.
- Migrations/ contém as migrations do Entity Framework Core, usadas para criar e atualizar a estrutura da base de dados ao longo do desenvolvimento.
- DependencyInjection/ contém a configuração dos serviços usados pela aplicação. É aqui que são registados os serviços, repositórios e outras dependências necessárias.
- Configuration/ contém classes ou configurações auxiliares usadas pela aplicação.
- Endpoints/ contém rotas HTTP específicas, como endpoints relacionados com autenticação.

De forma geral, a comunicação entre as camadas acontece da seguinte forma: as páginas Blazor chamam os serviços, os serviços aplicam a lógica de negócio e comunicam com os repositórios, e os repositórios acedem à base de dados através do AppDbContext.

Também existe integração com a API externa TMDB, usada para obter dados sobre filmes. A informação recebida em JSON é adaptada para os modelos internos da aplicação, permitindo usar dados como títulos, descrições, cartazes, classificações e metadados dentro do sistema.

Funcionalidades Implementadas

A aplicação MadFlix permite aos utilizadores explorar filmes e festivais de cinema através de uma plataforma web. A página inicial apresenta filmes recomendados e permite pesquisar filmes, consultar os seus detalhes, ver informações como género, ano, classificação, descrição, elenco e trailer.

No contexto do projeto, o "filme" é simulado através do trailer. Numa aplicação real de streaming, esta página de visualização corresponderia ao conteúdo completo do filme. Como este é um projeto académico, foi usado o trailer como forma de representar a reprodução do filme.

Cada filme possui uma página própria, onde o utilizador pode consultar mais informação, ver o trailer, comentar e avaliar com estrelas. Os comentários ficam associados ao filme e podem ser visualizados por outros utilizadores. Apenas administradores têm permissão para remover comentários.

A aplicação também permite consultar festivais disponíveis. Cada festival pode ter vários filmes associados, formando o seu catálogo. O utilizador pode abrir o catálogo de um festival e consultar os filmes que pertencem a esse festival. Os administradores podem criar novos festivais e adicionar filmes ao catálogo de um festival.

O sistema inclui diferentes tipos de acesso:

Bilhete de Sessão

O bilhete de sessão é usado para filmes individuais, ou seja, filmes que não dependem de um festival. Este bilhete dá acesso ao filme durante uma janela horária específica. No projeto, a sessão individual tem uma duração definida, permitindo ao utilizador ver o filme apenas enquanto o acesso estiver ativo.

Passe Diário

O passe diário é um acesso a um festival durante um determinado dia escolhido pelo utilizador. O utilizador só pode escolher uma data que esteja dentro do período do festival. Este passe não funciona como 24 horas a partir da compra, mas sim como acesso às sessões desse dia. As sessões implementadas são, por exemplo:

- Sessão 1: 11:30- 16:30
- Sessão 2: 18:00- 23:00

Se o utilizador tiver um passe diário válido, pode ver os filmes do catálogo do festival durante as sessões disponíveis desse dia.

Passe Completo

O passe completo dá acesso ao festival durante todos os dias em que o festival decorre. A lógica é semelhante à do passe diário, mas aplicada a todas as datas do festival. Assim, o utilizador pode aceder às sessões disponíveis em cada dia, desde a data de início até à data de fim do festival.

Aluguer Digital

O aluguer digital permite ao utilizador escolher uma duração em horas. Ao contrário do passe diário, este acesso começa no momento em que a compra é finalizada. O sistema calcula automaticamente quais partes das sessões ficam cobertas pela duração escolhida. Por exemplo, se o utilizador comprar 2 horas às 18:00, o acesso fica válido das 18:00 às 20:00, mesmo que a sessão completa vá até às 23:00.

As sessões têm estados visuais. Quando uma sessão está disponível no momento atual, aparece como disponível a verde. Quando está fora do horário válido, aparece como indisponível a vermelho. Esta lógica ajuda o utilizador a perceber rapidamente se pode ou não ver determinado conteúdo naquele momento.

A aplicação possui um carrinho lateral onde o utilizador pode adicionar acessos, consultar os itens escolhidos, ver o preço de cada item e o total da compra. O pagamento é simulado, permitindo escolher métodos como PayPal, MB Way, Cartão Bancário ou Apple Pay. Após a confirmação, o sistema simula o processamento do pagamento e ativa os acessos correspondentes.

Existe também uma página de Bilhetes, onde o utilizador pode consultar o histórico dos seus acessos. Os bilhetes são separados em três categorias:

- Ativos
- Pendentes
- Expirados

Os bilhetes ativos permitem aceder ao filme ou ao catálogo do festival. Os pendentes representam acessos comprados que ainda não começaram. Os expirados representam acessos cuja janela de visualização já terminou.

Nos acessos associados a festivais, o utilizador pode abrir o catálogo diretamente na página de bilhetes e ver os filmes disponíveis. Caso o administrador adicione novos filmes ao festival depois da compra, esses filmes também ficam acessíveis enquanto o utilizador tiver um passe válido, pois o acesso é associado ao festival e não apenas aos filmes existentes no momento da compra.

A aplicação inclui ainda uma área de perfil, onde os utilizadores podem criar listas pessoais e adicionar filmes a essas listas. Esta funcionalidade permite organizar filmes de interesse.

A aplicação inclui também um sistema de recomendações de filmes. Inicialmente existia uma zona de filmes recomendados, mas esta funcionalidade foi melhorada para apresentar recomendações mais reais e ajustadas ao utilizador, tornando a página inicial mais dinâmica e útil.

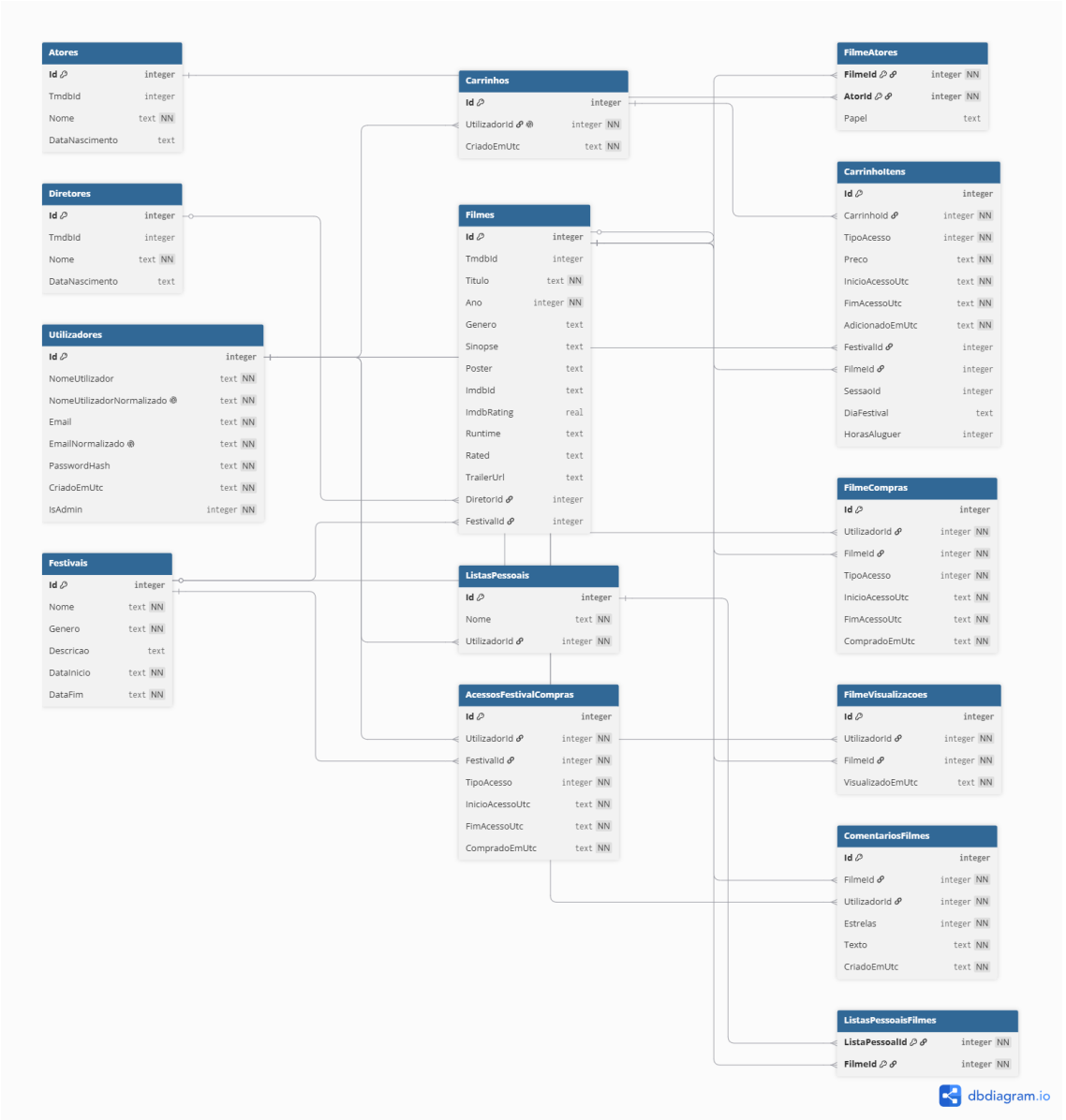
Na área de perfil, para além da consulta dos dados do utilizador e da gestão de listas pessoais, foi adicionada a possibilidade de alterar a imagem de perfil. Desta forma, cada utilizador pode personalizar a sua conta dentro da plataforma.

Foi ainda implementada uma funcionalidade de votação nos festivais. Os utilizadores podem votar no melhor filme de um festival, permitindo criar uma componente de participação e interação dentro do catálogo de cada festival. Esta funcionalidade aproxima o sistema de uma experiência real de festival, onde o público pode demonstrar preferência por determinados filmes.

Em resumo, o MadFlix permite pesquisar filmes, consultar detalhes, ver trailers, comentar, avaliar, criar listas pessoais, explorar festivais, comprar acessos, simular pagamentos, consultar bilhetes e visualizar conteúdos de acordo com as regras de acesso definidas.

Esquema Relacional da Base de Dados

Este diagrama representa a estrutura da base de dados do MadFlix e mostra as principais entidades do sistema e as relações entre elas. Nele é possível ver como os utilizadores se relacionam com carrinhos, compras, bilhetes, comentários, visualizações e listas pessoais, bem como a ligação entre filmes, festivais, atores e diretores. O diagrama ajuda a compreender a organização dos dados e a forma como a aplicação guarda a informação necessária para gerir filmes, festivais, acessos e interações dos utilizadores.



Padrões de Desenho Aplicados

Foram aplicados vários padrões de desenho no projeto.

O padrão Factory Method / Abstract Factory foi usado na criação dos diferentes tipos de acesso. A AcessoFactory é responsável por criar objetos
como BilheteSessao, PasseDiario, PasseCompleto e AluguerDigital, evitando que a lógica de criação fique espalhada pelo código.

O padrão Decorator foi usado no aluguer digital. A duração base do aluguer pode ser alterada através de um decorator, permitindo modificar o comportamento do acesso sem alterar diretamente a classe base.

O padrão State foi usado em duas zonas importantes. Primeiro, no pagamento, para representar estados como processamento, finalização e compra concluída. Segundo, nas sessões, para representar se uma sessão está disponível ou indisponível. Isto permite que a interface mostre as sessões a verde ou vermelho conforme o estado atual.

O padrão Strategy foi usado nos métodos de pagamento. Cada método, como MB Way, PayPal, Cartão Bancário ou Apple Pay, tem a sua própria estratégia de processamento simulado.

O padrão Observer foi usado na parte dos festivais e notificações. Quando um festival é criado ou um filme é adicionado a um festival, os observadores podem ser notificados. Isto permite desacoplar o evento da ação que acontece depois.

O padrão Repository foi usado para separar o acesso aos dados da lógica de negócio. Assim, os serviços não trabalham diretamente com o Entity Framework Core, mas sim com interfaces de repositórios.

Também foi usado um estilo próximo de Adapter na integração com a API TMDB. A API externa disponibiliza dados em JSON, e o sistema adapta esses dados para os modelos internos da aplicação.